

1 The A* search algorithm

The A* search algorithm solves the shortest path problem for weighted graphs. On graphs with non-negative edge weights, it is guaranteed to terminate and it is complete. A* can be seen as an expansion of Dijkstra's algorithm, which we will explain in the following paragraph. Once Dijkstra's algorithm is understood, the transition to A* is rather simple.

1.1 Dijkstra's algorithm for shortest paths

Dijkstra's algorithm solves the shortest path problem for a graph with non-negative edge weights. Given a starting node, the algorithm returns the shortest path and its distance for every node in the graph. Of course, one can also limit the algorithm to halt once a given goal node has been found. This is the approach we want to follow.

We are encoding the graph as one table. An example is shown in Table 1. We will use this graph for a running example to explain Dijkstra's algorithm.

1.1.1 Query Start

For both variants, we first define two macros for the start- and goal-node respectively.

```
CREATE MACRO start_node() AS {start_node};  
CREATE MACRO goal_node() AS {goal_node};
```

We can see in Listing 1 that the two CTE versions are almost identical at the start. We define the following variables:

- `node_id`, identifying the expanded node for this iteration;
- `dist`, the shortest distance from the start node to the expanded node;
- `f`, which is equal to `dist` for Dijkstra's algorithm, but a bit more complicated for A* (see Section 1.2);
- `prev`, the id of the node that leads back to the start node the fastest;
- `visited`, a boolean flag stating whether the node has already been visited.

In the initial case, the only node known to us is the start node, with a distance from the start node of `dist = 0`, no previous node `prev = NULL`, and the visited flag set to `visited = false`.

For the dict-based CTE, we use `node_id` as key.

1.1.2 Recursive Step: dict-based CTE

The recursive step is the important part of the query. We shall see later that the table-based version can be written as an extension of the dict-based CTE, which is why we start with the latter. Listing 2 shows the relevant code. It is a CTE that can be separated into three logical parts. Notice that, in every part, we select from the recurring table.

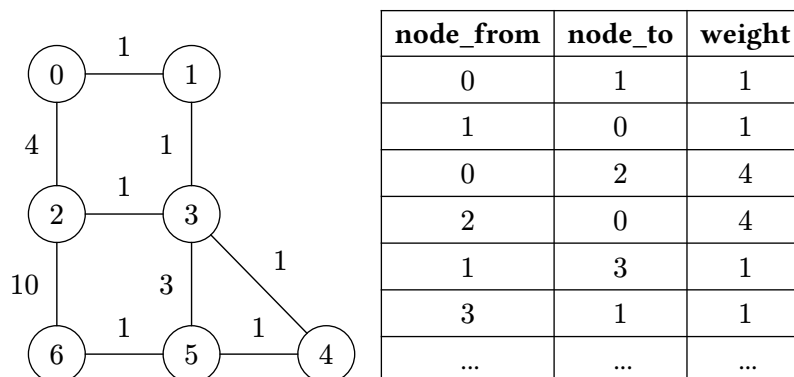


Table 1: Example graph (left) and the first rows of its table representation (right)

```

WITH RECURSIVE dijkstra (
    node_id,
    dist,
    f,
    prev,
    visited
) AS (
    SELECT
        start_node(),
        0,
        0,
        NULL,
        false

    UNION ALL

    (
        -- <recursive step>
    )
)
-- <outer query>

```

```

WITH RECURSIVE dijkstra (
    node_id,
    dist,
    f,
    prev,
    visited
) USING KEY (node_id) AS (
    SELECT
        start_node(),
        0,
        0,
        NULL,
        false

    UNION

    (
        -- <recursive step>
    )
)
-- <outer query>

```

Listing 1: Initial case of Dijkstra’s algorithm for both recursive CTE types in comparison.

node_id	dist	f	prev	visited
0	0	0	None	false

1st iteration

node_id	dist	f	prev	visited
0	0	0	None	true
1	1	1	0	false
2	4	4	0	false

2nd iteration

node_id	dist	f	prev	visited
0	0	0	None	true
1	1	1	0	true
2	4	4	0	false
3	2	2	1	false

3rd iteration

node_id	dist	f	prev	visited
0	0	0	None	true
1	1	1	0	true
2	3	3	3	false
3	2	2	1	true
4	3	3	3	false
5	5	5	3	false

4th iteration

Table 2: Recurring table after step ❶ in the first four iterations. The selected node for each iteration is marked in green.

❶st, we select one single node from the recurring table and bind its id to `min_node(id)`. This node has to meet two criteria, as can be seen in the `WHERE` clause: First, it must not be visited. Second, its `f`-value must be minimal. In the first iteration, the only node in the recurring table is the start node.

❷nd step: We want so visit each node at most once. For this reason, we introduced the `visited` variable at the query start. The `node_id` of the node we are currently visiting has been bound to `min_node(id)` in ❶. Thus, we should update its `visited` value to `true`. Keep in mind that we are

node_id	dist	f	prev	visited
0	0	0	None	true

1st iteration

node_id	dist	f	prev	visited
0	0	0	None	true
1	1	1	0	true
2	4	4	0	false
3	2	2	1	true

3rd iteration

node_id	dist	f	prev	visited
0	0	0	None	true
1	1	1	0	true
2	4	4	0	false

2nd iteration

node_id	dist	f	prev	visited
0	0	0	None	true
1	1	1	0	true
2	3	3	3	false
3	2	2	1	true
4	3	3	3	true
5	5	5	3	false

4th iteration

Table 3: Recurring table after step ② in the first four iterations. The changed value for each iteration is marked in orange.

currently working with the dict-based CTE. Because we set the key to node_id at the start, when updating the values for the currently visited node, the values get updated in the dictionary.

③rd step: The goal of this step is to take all neighbors of min_node and update their dist-values, if it turns out that the path over min_node is shorter.

First, notice that the first line of the FROM-clause and the first two lines of the WHERE-clause are identical to ②, the only difference being that we bind mind_node to the name sml (standing for “smallest”) here. The second line in the FROM-clause binds the graph to the name nbs (“neighbors”), on the condition sml.node_id = nbs.node_from. With this, we can select the node_ids of all neighbors of min_node by selecting nbs.node_to. We immediately use this in the third line of the FROM-clause, where we bind the recurring table to the name old, on the condition nbs.node_to = old.node_id. With this, we have access to the values of smallest’s neighbors, by selecting old. Notice that we used a LEFT OUTER JOIN to join old, which returns NULL-values for neighbors which are not part of the dictionary yet.

With this preparation, we are able to understand the third line of the WHERE-clause. This line is the heart of Dijkstra’s algorithm. It defines the final condition on which values are being selected. Rewriting the coalesce-function in a mathematical way, it looks like this:

$$\text{coalesce}(a, b) = \begin{cases} a & \text{if } a \neq \text{NULL} \\ b & \text{else} \end{cases}$$

old.dist can be NULL, as explained in the previous paragraph. In this case, the condition is true by default. Otherwise we evaluate the condition, sml.dist + nbs.weight < old.dist. Only nodes for which this condition is true, i.e. nodes for which the path over min_node is smaller than their current shortest path, will be selected in this step.

node_id	dist	f	prev	visited
0	0	0	None	true
1	1	1	0	false
2	4	4	0	false

1st iteration

node_id	dist	f	prev	visited
0	0	0	None	true
1	1	1	0	true
2	4	4	0	false
3	2	2	1	false

2nd iteration

node_id	dist	f	prev	visited
0	0	0	None	true
1	1	1	0	true
2	3	3	3	false
3	2	2	1	true
4	3	3	3	false
5	5	5	3	false

3rd iteration

node_id	dist	f	prev	visited
0	0	0	None	true
1	1	1	0	true
2	3	3	3	false
3	2	2	1	true
4	3	3	3	true
5	4	4	4	false

4th iteration

Table 4: Recurring table after step ❶ in the first four iterations. Newly added nodes are marked in green, while modified values are marked in orange.

Finally, we examine the SELECT-clause. With the conditions mentioned above, we have just found a set of neighbors which either have never been selected before, or for which we found a shorter path. We now set or update those neighbor's entries in the dictionary, respectively.

- `nbs.node_to` is the neighbor's `node_id`, as explained above.
- as `sml.dist + nbs.weight` is the shortest path currently known to reach this neighbor, we set both `dist` and `f` to this value.
- as we found this smallest path by going over `min_node`, we set its id, `sml.node_id`, to be previous node `prev` in the path to reach the neighbor.
- the neighbor has still not been visited yet.

1.1.3 Recursive Step: table-based CTE

The union-table-based version of A* can be viewed as an extension of the dict-based version. We follow the same approach, however, because we can only access values which have been calculated in the previous iteration, we have to carry all values manually. This vastly increases the size of the union table.

Listing 3 shows the relevant code. The dict-based recursive step from Listing 2 can be inserted at the marked spot if replacing every occurrence of `recurring.table` with `filtered_dijkstra`.

Additionally, two code blocks are required.

❶st, as multiple occurrences of the same key can appear in the union table, we need to specify which one should be selected. This is what the CTE `filtered_dijkstra` does. To decide which values to select and which to discard, we can use the nature of Dijkstra's algorithm:

1. A new entry for a given node is only being generated if a smaller distance to this node has been found. Thus, we should select `min(dist)` and `argmin(f, dist)`, `argmin(prev, dist)`, respectively.

```

-- ❶ Finding the id of the node with minimal distance
WITH min_node(id) AS (
    SELECT node_id
    FROM recurring.astar
    WHERE
        NOT visited AND
        f = (SELECT min(f) FROM recurring.astar WHERE NOT visited)
    LIMIT 1
)

-- ❷ Setting visited = true for the minimal node
SELECT
    node_id,
    dist,
    f,
    prev,
    true
FROM recurring.astar
WHERE
    node_id = (SELECT id FROM min_node) AND
    node_id != goal_node()

UNION

-- ❸ Updating the neighbors of the minimal node
SELECT
    nbs.node_to,
    sml.dist + nbs.weight,
    sml.dist + nbs.weight,
    sml.node_id,
    false
FROM
    recurring.astar AS sml JOIN
    {graph}          AS nbs ON sml.node_id = nbs.node_from LEFT OUTER JOIN
    recurring.astar AS old ON nbs.node_to = old.node_id
WHERE
    sml.node_id = (SELECT id FROM min_node) AND
    sml.node_id != goal_node() AND
    sml.dist + nbs.weight < coalesce(old.dist, CAST('inf' AS FLOAT))

```

Listing 2: Recursive step of Dijkstra's algorithm for the dict-based CTE

2. If a node has been marked as visited, it will never be selected again. Thus, if one instance of the node has its visited-value set to true, we should select true overall - which can simply be implemented by `bool_or(visited)`.

❷ndly, after all other values have been selected, we need to carry to rest of the table in order to not lose any information. For this, we simply select the entire `filtered_dijkstra`-table. The `WHERE`-clause implements the break condition.

1.2 Expanding Dijkstra with a heuristic function

We will now make the step from Dijkstra's algorithm to A*. So far, the only information we had about the goal node was its `node_id`. In many use cases, however, information is available about the direction in which the goal is located.

A vivid example is that of route planning. When travelling, say, via train, we can depict the railway network as a graph, with stations being the nodes, railroads being the edges and travel times or

```

-- ❶ Group equal nodes
WITH filtered_dijkstra (
    node_id,
    dist,
    f,
    prev,
    visited
) AS (
    SELECT
        node_id,
        min(dist),
        argmin(f, dist),
        argmin(prev, dist),
        bool_or(visited)
    FROM astar
    GROUP BY node_id
),

-- <dict-based recursive step>

UNION

-- ❷ Carry table
SELECT *
FROM filtered_dijkstra
WHERE (SELECT id FROM min_node) != goal_node()

```

Listing 3: Recursive step of Dijkstra’s algorithm for the table-based CTE, based on the dict-based version.

distances being the edge weights. We then want to find the fastest route from our current location, the start node, to our destination, the goal node.

Dijkstra’s algorithm approaches this problem by searching in *every* direction until it found the destination. While it always finds the best solution, the runtime is obviously not optimal, as we constantly calculate shortest paths for stations which we know to be in the wrong direction.

This problem can be solved by using a *heuristic function*. In our example, a heuristic h for a node n would simply be the air-line distance between a node and the goal-node,

$$h(n) = \text{dist}(n, \text{goal_node}).$$

There are many possibilities to define a heuristic function. In order for A* to return an optimal solution, and to have an optimal runtime, the heuristic function has to meet two criteria.

1. It has to be *admissible*, i.e. it must never overestimate the cost of reaching the goal node, and
2. It has to be consistent, i.e. for every node n and for every neighbor p of n , the following formula for the heuristic function h must apply:

$$h(n) \leq w(n, p) + h(p),$$

where $w(n, p)$ is the weight of the edge between nodes n and p .

Let us now apply this to our query. Assuming a heuristic function has been defined via a macro `CREATE MACRO h(n) AS ...`, we update our query by applying $f(n) = \text{dist}(n) + h(n)$. The complete dict-based query is shown in Listing 4.

TODO: Maybe move the full query to the appendix or something. Or just say which lines to change.

```

CREATE MACRO start_node() AS {start_node};
CREATE MACRO goal_node() AS {goal_node};
CREATE MACRO h(n) AS -- <heuristic function>;

WITH RECURSIVE astar (
    node_id,
    dist,
    f,
    prev,
    visited
) USING KEY (node_id) AS (
    SELECT
        start_node(),
        0,
        h(start_node()),
        NULL,
        false

    UNION

    (
        WITH min_node(id) AS (
            SELECT node_id
            FROM recurring.astar
            WHERE
                NOT visited AND
                f = (SELECT min(f) FROM recurring.astar WHERE NOT visited)
            LIMIT 1
        )
        SELECT
            node_id,
            dist,
            f,
            prev,
            true
        FROM recurring.astar
        WHERE
            node_id = (SELECT id FROM min_node) AND
            node_id != goal_node()

        UNION

        SELECT
            nbs.node_to,
            sml.dist + nbs.weight,
            sml.dist + nbs.weight + h(nbs.node_to),
            sml.node_id,
            false
        FROM
            recurring.astar AS sml JOIN
            {graph} AS nbs ON sml.node_id = nbs.node_from LEFT OUTER JOIN
            recurring.astar AS old ON nbs.node_to = old.node_id
        WHERE
            sml.node_id = (SELECT id FROM min_node) AND
            sml.dist + nbs.weight < coalesce(old.dist, CAST('inf' AS FLOAT)) AND
            sml.node_id != goal_node()
    )
),
-- <outer query>

```

Listing 4: A* as a dict-based CTE